

Express

- **Express.js** is a minimal and flexible **Node.js web application framework**.
- It simplifies building **web servers and APIs**.
- Built on top of Node.js, it handles **HTTP requests, routing, middleware**, and more.
- Essentially, it makes Node.js easier to work with for **backend development**.

Setting up Express.js

1. Initialize a Node.js project:

```
mkdir myapp  
cd myapp  
npm init -y
```

2. Install Express:

```
npm install express
```

3. Create a basic server:

```
// index.js  
const express = require('express');  
const app = express();  
  
// Middleware (optional)  
app.use(express.json()); // parse JSON body  
  
// Routes  
app.get('/', (req, res) => {  
  res.send('Hello World!');  
});  
  
// Start server
```

```
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

4.Run server:

`node index.js`

Go to `http://localhost:3000` → You'll see **Hello World!**

1. The Request (req) Object

The req object represents the **HTTP request** and contains data sent by the client (browser, Postman, frontend, etc.).

Key Properties

Property	Description
req.params	Route parameters from the URL (/users/:id)
req.query	Query string parameters (/search?term=node)
req.body	Data sent in the body of POST/PUT requests (requires middleware like <code>express.json()</code>)
req.headers	HTTP headers of the request
req.method	HTTP method (GET, POST, etc.)
req.url	Requested URL
req.cookies	Cookies sent by the client (requires <code>cookie-parser</code>)
req.ip	IP address of the client
req.get('header-name')	Access a specific header

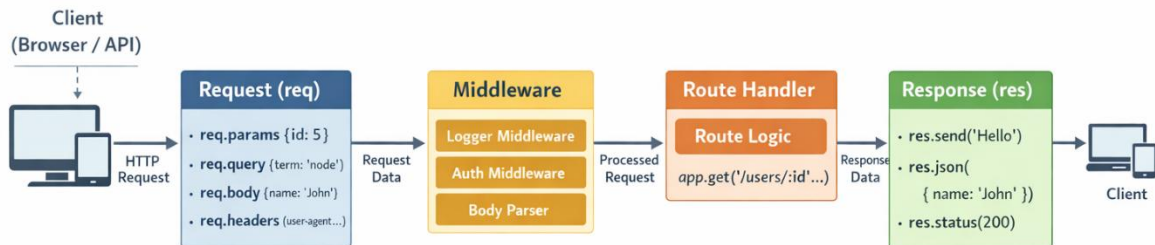
The Response (res) Object

The res object is used to **send data back to the client**.

Key Methods

Method	Description
res.send()	Send a response (string, object, array)
res.json()	Send JSON data (automatically sets Content-Type: application/json)
res.status(code)	Set HTTP status code
res.redirect(url)	Redirect client to another URL
res.set(header, value)	Set HTTP headers
res.cookie(name, value)	Set cookies (requires cookie-parser)
res.sendFile(path)	Send a file as response

Express.js Request-Response Cycle



```
app.post('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  const data = req.body; // e.g., { name: 'New User' }  
  
  console.log(`Updating user ${userId} with data:`, data);  
  
  res.status(200).json({  
    message: `User ${userId} updated successfully`,  
    updatedData: data  
  });  
});
```

Here, **request data comes from both req.params and req.body.**

Response is a JSON object with a message and the updated data.

App.use() and Middleware

app.use() is used to **register middleware functions** in Express.js.

Middleware functions are executed in the **order they are defined** during the request-response cycle.

They can perform tasks like:

- Logging requests
- Parsing request bodies
- Handling authentication
- Serving static files
- Handling errors

Syntax:

`app.use([path], callback)`

- path (optional): the route where the middleware applies. Defaults to / (all routes).
- callback: the middleware function, which takes (req, res, next).

3. Examples of app.use()

3.1 Global Middleware (applies to all routes)

```
app.use((req, res, next) => {  
  console.log(`${req.method} ${req.url}`);  
  next(); // move to next middleware or route  
});
```

- Every request will log the method and URL before going to the route handler.

3.2 Middleware for Specific Path

```
app.use('/admin', (req, res, next) => {  
  console.log('Admin area access attempt');  
  next();  
});
```

```
app.get('/admin/dashboard', (req, res) => {  
  res.send('Admin Dashboard');  
});
```

- Only requests starting with /admin trigger this middleware.

3.3 Using Built-in Middleware

- Express has **built-in middleware** like `express.json()` and `express.static()`:

```
// Parse JSON body  
app.use(express.json());
```

```
// Serve static files from 'public' folder  
app.use(express.static('public'));
```

- Example: A POST request with JSON body will automatically populate `req.body`.

Need of `app.use(express.json())`..?

In Express.js, `app.use(express.json())` is a built-in middleware that parses incoming requests with a JSON payload and converts it into a JavaScript object accessible via `req.body`. Without it, `req.body` for JSON requests would be undefined, so the server cannot read the client's data. It must be placed **before routes** that handle JSON requests and only processes requests with Content-Type: `application/json`. This middleware is essential for building RESTful APIs that receive JSON data from clients.

3.4 Using Third-party Middleware

```
const morgan = require('morgan'); // logging library
```

```
app.use(morgan('dev')); // logs requests in dev format
```

3.5 Error-handling Middleware

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});
```

Routing

Routing defines **how an application responds to client requests** to a particular **endpoint (URL)** and **HTTP method**.

1. Basic Route

A basic route handles a **specific HTTP method** and **path**.

```
app.get('/', (req, res) => {  
  res.send('Welcome to Home Page');  
});
```

```
app.post('/submit', (req, res) => {  
  res.send('Form submitted');  
});
```

- `app.get()` → handles GET requests
- `app.post()` → handles POST requests
- `res.send()` → sends a response to the client

Tip: For APIs, `res.json()` is often preferred over `res.send()`.

2. Route Parameters (Dynamic Routes)

- Dynamic segments in the URL are **parameters** starting with **:**.
- Useful for **user profiles, product pages, or IDs**.

```
app.get('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  res.send(`User ID is ${userId}`);  
});
```

- `/users/5` → Response: User ID is 5

Multiple Parameters

```
app.get('/users/:userId/books/:bookId', (req, res) => {  
  res.send(req.params);  
});  
// /users/5/books/20 → { userId: '5', bookId: '20' }
```

3. Query Parameters

- Extra info sent in the URL after `?`.
- Accessible via `req.query`.

```
app.get('/search', (req, res) => {  
  const { term, page } = req.query;  
  res.send(`Searching for "${term}" on page ${page}`);  
});  
// /search?term=node&page=2
```

- Great for **filtering, pagination, or search queries**.

4. Modular Routing with Router IMP

For large apps, **separate routes into modules**.

```
const express = require('express');  
const router = express.Router();
```



```
// Example routes
router.get('/', (req, res) => res.send('Users Home'));
router.get('/:id', (req, res) => res.send(`User ${req.params.id}`));

module.exports = router;
```

Use it in main app:

```
const usersRouter = require('./users');
app.use('/users', usersRouter);
```

- Now /users routes are **handled in their own module** → keeps code clean.

5. Middleware in Routes

- Middleware can be **global**, **route-specific**, or **router-specific**.

```
function authMiddleware(req, res, next) {
  const loggedIn = true; // just an example
  if (loggedIn) next();
  else res.status(401).send('Unauthorized');
}
```

```
app.get('/dashboard', authMiddleware, (req, res) => {
  res.send('Dashboard');
});
```

Only authenticated users can access /dashboard.

Routing in Express.js is the mechanism to handle client requests to specific endpoints using different HTTP methods. Routes can be static (/about), dynamic (/users/:id), or even pattern-based (/ab*cd). Middleware can be chained for authentication, logging, or validation. For large applications, modular routing using `express.Router()` keeps code maintainable. Correct use of HTTP methods (GET, POST, PUT, PATCH, DELETE) aligns with RESTful best practices.

Environment Variables in Express.js

Environment variables are **variables that store configuration outside of your code**. They allow your app to be **flexible, secure, and environment-specific** (development, staging, production).

1. Why Use Environment Variables?

1. Security

Don't hardcode secrets like API keys, database passwords, JWT secrets.

2. Flexibility

- Easily change configuration without modifying code.
- Example: PORT, NODE_ENV, database URLs, API keys.

3. Environment-specific configuration

- Development vs production vs staging.

2. Accessing Environment Variables in Node.js

Using .env File with dotenv

.env is a file where you define environment variables for your project.

Step 1: Install dotenv

```
npm install dotenv
```

Step 2: Create a .env file in your project root

```
PORT=5000
DB_URL=mongodb://localhost:27017/myapp
JWT_SECRET=mySuperSecretKey
NODE_ENV=development
```

Step 3: Load .env in your app

```
require('dotenv').config();

const express = require('express');
const app = express();

const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => res.send('Hello World'));

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

- `require('dotenv').config()` → loads .env into `process.env`.

4. Best Practices for dotenv

1. Do not commit .env to Git

- Add .env to .gitignore.
- Keeps secrets safe.

2. Use .env.example

Share variable names without actual secrets for other developers:

```
PORT=
DB_URL=
JWT_SECRET=
NODE_ENV=
```

3. Type Conversion

Variables from .env are always **strings**. Convert if needed:

```
const PORT = parseInt(process.env.PORT, 10) || 3000;
const DEBUG = process.env.DEBUG === 'true';
```

Environment-specific .env files

- .env.development, .env.production, .env.test
- Use a library like **dotenv-flow** for multi-env management.

Environment variables store configuration data outside the code, like ports, API keys, database URLs, and secrets. Using them improves security, flexibility, and allows environment-specific settings (development, production, staging). In Express.js, dotenv is a popular library that loads variables from a .env file into process.env, so your app can access them safely without hardcoding sensitive information.

Input validation

Input validation is verifying that the data coming from a client (request body, query params, headers) matches the expected format and rules.

Prevents **bad data**, **security issues**, and **server crashes**.

Common validation examples:

- Required fields (e.g., name cannot be empty)
- Data types (e.g., age must be a number)
- Email format
- Password strength
- Query parameters or route params

Input validation is essential in backend development to ensure **security, data integrity, and application stability**. It prevents malicious or malformed data from crashing the app, protects against attacks, and enforces business rules. In Express.js, validation ensures that requests contain the correct types, formats, and required fields before reaching route handlers or the database, improving API reliability and maintainability.

1. Traditional Validation (Manual)

```
app.post('/register', (req, res) => {
  const { name, email, age } = req.body;

  if (!name || !email) {
    return res.status(400).json({ error: 'Name and Email are required' });
  }

  if (typeof age !== 'number') {
    return res.status(400).json({ error: 'Age must be a number' });
  }

  res.json({ message: 'User registered', data: req.body });
});
```

✅ Works, but **lots of repetitive code** for multiple routes.

Using Validation Libraries

- Libraries like **Joi**, **Yup**, **Zod** allow **declarative validation schemas**.
- **Zod** is **TypeScript-first**, fast, and developer-friendly.

Zod for Input Validation

Step 1: Install Zod

```
npm install zod
```

Step 2: Define a Schema

```
const { z } = require('zod');

const userSchema = z.object({
  name: z.string().min(1, "Name is required"),
  email: z.string().email("Invalid email"),
  age: z.number().int().positive("Age must be positive"),
});
```

- `z.object({...})` → defines expected object shape
- `.min()`, `.email()`, `.int()`, `.positive()` → validators
- **Error messages** can be customized

Step 3: Validate Input in Route

```
app.post('/register', (req, res) => {
  try {
    const validatedData = userSchema.parse(req.body);
    // If validation passes, validatedData contains properly typed object
    res.json({ message: 'User registered', data: validatedData });
  } catch (error) {
    res.status(400).json({ errors: error.errors }); // send validation errors
  }
});
```

- `userSchema.parse(data)` → throws if invalid
- No more manual if-else checks

Step 4: Optional Fields & Defaults

```
const updateSchema = z.object({
  name: z.string().optional(),
  email: z.string().email().optional(),
  age: z.number().optional(),
});
```

- `.optional()` → field is not required
- `.default(value)` → provide default if missing

Step 5: Integrating Zod as Middleware

```
function validate(schema) {
  return (req, res, next) => {
    try {
      req.body = schema.parse(req.body);
      next();
    } catch (err) {
      res.status(400).json({ errors: err.errors });
    }
  }
}
```

```
}  
};  
}
```

```
app.post('/register', validate(userSchema), (req, res) => {  
  res.json({ message: 'User registered', data: req.body });  
});
```

- Makes validation **reusable for multiple routes**
- Keeps routes clean and maintainable

Authentication and Authorization

Authentication → “Are you who you say you are?”

Authorization → “Are you allowed to do this?”

Hashing

Hashing is a **one-way process** that converts input data of any size into a **fixed-length string** (hash).

Key Points:

1. **Irreversible:** Once hashed, you **cannot get back the original data**.
2. **Deterministic:** Same input always produces the same hash.
3. **Fixed-length output:** No matter how long the input is, the hash length is constant.
4. **Used for verification, not secrecy:**
 - Example: passwords, file integrity checks (checksums).

Example:

Input: myPassword123

SHA-256 hash: ef92b778bafe771e89245b89ecbc1d2d

- Even a small change in input produces a **completely different hash** (avalanche effect).
- Libraries: bcrypt, SHA-256, SHA-512.

Use Case in Web Apps:

- Passwords → never store plaintext, only store the hash.
- API tokens → hash to verify validity without exposing the secret.

Encryption

Encryption is a reversible process that converts plaintext into ciphertext using an algorithm and a key.

Key Points:

1. Reversible: Using the correct key, the original data can be recovered.
2. Variable-length output: Ciphertext length can vary depending on algorithm.
3. Used for secrecy: Protects data while storing or transmitting.
4. Key-dependent: Without the key, decryption is not possible.

Example:

Plaintext: myPassword123

AES-256 Encrypted: 3f5c2d1a9b7f4e6d...

Decrypt with key → myPassword123

- Libraries: crypto (Node.js), AES, RSA.

Use Case in Web Apps:

- Sensitive messages, files, or database columns that need to be recovered later.
- Example: encrypting credit card info, personal documents.
- **Hashing: Like grinding a sugar cube into powder → you can't reconstruct the cube, but you can check if two powders came from the same cube.**

- **Encryption:** Like locking your sugar cube in a safe → you can open it later with a key.

Hashing is a one-way function that converts data into a fixed-length string, useful for storing passwords or verifying integrity, and cannot be reversed. **Encryption** is a reversible process that transforms data into ciphertext using a key, allowing it to be decrypted later, making it suitable for sensitive information that needs recovery.

Q. Why not store Plain password in Dbs..?

Storing plain passwords in the database is extremely unsafe because if the database is compromised, all user credentials are immediately exposed. It also poses insider risks, violates privacy regulations, and encourages bad practices. Instead, passwords should always be hashed (e.g., using bcrypt) before storing, ensuring that even if the database is leaked, original passwords cannot be recovered.

Q. when we forgot password we have to create new password and not get old password back

Modern applications never store plain passwords; they store only hashed values. Hashing is a one-way process, so the original password cannot be retrieved. Therefore, when a user forgets their password, systems cannot send it back. Instead, a secure password reset flow is used, allowing the user to create a new password while keeping security and privacy intact.

JWT JsonWebToken

JWT (JSON Web Token) is a compact, URL-safe token used for securely transmitting information between parties. It is signed so the server can verify its authenticity and optionally encrypted for confidentiality.

Key Points:

- **Stateless authentication:** The server doesn't need to store sessions; all info is in the token.
- **Self-contained:** JWT carries information like user id, role, expiry.
- **Signed:** Using a secret or private key → prevents tampering.

2. Structure of JWT

A JWT has **3 parts** separated by dots:

`header.payload.signature`

1) Header

- Contains metadata about the token: type (JWT) and signing algorithm (HS256).

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

2) Payload

- Contains the actual data (claims), e.g., user info, role, expiration.

```
{  
  "id": 1,  
  "email": "user@example.com",  
}
```

```
"role": "admin",  
"exp": 1710000000  
}
```

- exp → expiration timestamp

3) Signature

- Created using:

HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)

- Ensures token is **authentic** and **untampered**.

3. Why JWT is Used

1. **Stateless authentication:** No need to store session on server.
2. **Scalable:** Ideal for **distributed systems / microservices**.
3. **Compact & portable:** Can be sent via headers, URL, cookies.
4. **Supports role-based access:** Payload can store role for authorization.
5. **Secure:** Signature ensures integrity (if using secret or private key).

4. JWT in Express.js – Implementation

Step 1: Install JWT

```
npm install jsonwebtoken
```

Step 2: Generate JWT on Login

```
const jwt = require('jsonwebtoken');
```

```
app.post('/login', (req, res) => {  
  const user = { id: 1, email: 'user@example.com', role: 'user' };  
  const secret = process.env.JWT_SECRET || "supersecret";
```

```
// Generate token
```

```
const token = jwt.sign(user, secret, { expiresIn: '1h' });
```

```
res.json({ message: "Login successful", token });
});
```

- `jwt.sign(payload, secret, options)` → creates a token
- `expiresIn` → token expiration

Step 3: Verify JWT in Middleware

```
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Bearer TOKEN

  if (!token) return res.sendStatus(401); // Unauthorized

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403); // Forbidden
    req.user = user; // attach user info
    next();
  });
}

// Protect routes
app.get('/dashboard', authenticateToken, (req, res) => {
  res.send(`Hello ${req.user.email}, welcome to your dashboard`);
});
```

Middleware checks the **token validity** before allowing access to protected routes.

Supports **authorization**: you can check `req.user.role` for role-based access.

Keep JWT secret safe → `.env` variable

Set short expiration time → 15m–1h; use refresh tokens for long sessions

Use HTTPS → tokens should never go over HTTP

Avoid storing sensitive info in payload → JWT payload is **base64 encoded but not encrypted**

Blacklist or rotate tokens → optional for logout or compromised tokens

JWT (JSON Web Token) is a stateless authentication method where the server issues a signed token containing user info after login. The client stores the token and sends it with each request. The server verifies the signature to authenticate the user and optionally checks roles for authorization. Unlike sessions, JWT does not require server-side storage, making it scalable and ideal for distributed systems. Security best practices include short expiry, HTTPS, and never storing sensitive data in the payload.

JWT can be decoded by anyone because the header and payload are Base64 encoded, not encrypted. However, decoding does not guarantee authenticity. Only the server with the secret key can verify the token's signature. Verification ensures that the token was issued by the server and has not been tampered with. Therefore, JWT security relies on signature verification, not on hiding the payload.